

Лабораторная работа №2

**Исследование протокола TCP и алгоритма управления очередью
RED**

Чемоданова Ангелина Александровна

Содержание

1	Цель работы	4
2	Задание	5
3	Выполнение лабораторной работы	6
3.1	Пример с дисциплиной RED.	6
3.2	Изменения в модели на узле s1 типа протокола TCP с Reno на NewReno, затем на Vegas.	9
3.3	Изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).	13
4	Выводы	15
	Список литературы	16

Список иллюстраций

3.1	Скрипт модели с дисциплиной RED	7
3.2	Скрипт модели с дисциплиной RED	8
3.3	График динамики размера окна TCP(сверху) и график динамики длины очереди и средней длины очереди(снизу) при типе протокола TCP Reno на узле s1	9
3.4	Скрипт изменений на узле s1 типа протокола TCP с Reno на Newreno	10
3.5	График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP NewReno на узле s1	10
3.6	Скрипт изменений на узле s1 типа протокола TCP с Reno на Vegas .	11
3.7	График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP Vegas на узле s1	12
3.8	Изменение процедуры finish	13
3.9	Изменение мониторинга размера окна TCP	13
3.10	Результаты изменений отображения окон с графиками	14

1 Цель работы

Исследовать протокол TCP и алгоритм управления очередью RED.

2 Задание

1. Рассмотреть пример с дисциплиной RED.
2. Изменить в модели на узле s1 тип протокола TCP с Reno на NewReno, затем на Vegas. Сравнить и пояснить результаты.
3. Внести изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

3 Выполнение лабораторной работы

3.1 Пример с дисциплиной RED.

Постановка задачи. Описание моделируемой сети: – сеть состоит из 6 узлов; – между всеми узлами установлено дуплексное соединение с различными пропускной способностью и задержкой 10 мс; – узел r1 использует очередь с дисциплиной RED для накопления пакетов, максимальный размер которой составляет 25; – TCP-источники на узлах s1 и s2 подключаются к TCP-приёмнику на узле s3; – генераторы трафика FTP прикреплены к TCP-агентам.

Первая часть скрипта модели с дисциплиной RED. (рис. 3.1)

```

# создание объекта Simulator
set ns [new Simulator]

# открытие на запись файла out.nam для визуализатора nam
set nf [open out.nam w]

# все результаты моделирования будут записаны в переменную nf
$ns namtrace-all $nf

# открытие на запись файла трассировки out.tr
# для регистрации всех событий
set f [open out.tr w]

# все регистрируемые события будут записаны в переменную f
$ns trace-all $f

# процедура finish закрывает файлы трассировки
# и запускает визуализатор nam
# Процедура finish:

proc finish {} {
    global tchan_
    # подключение кода AWK:
    set awkCode {
        {
            if ($1 == "Q" && NF>2) {
                print $2, $3 >> "temp.q";
                set end $2
            }
            else if ($1 == "a" && NF>2)
                print $2, $3 >> "temp.a";
        }
    }
    set f [open temp.queue w]
    puts $f "TitleText: red"
    puts $f "Device: Postscript"
    if { [info exists tchan_] } {
        close $tchan_
    }
    exec rm -f temp.q temp.a
    exec touch temp.a temp.q
    exec awk $awkCode all.q
    puts $f "\"queue
    exec cat temp.q >@ $f
    puts $f "\\n\\\"ave_queue
    exec cat temp.a >@ $f
    close $f
    # Запуск xgraph с графиками окна TCP и очереди:
    exec xgraph -bb -tk -x time -t "TCPReNoCWND" WindowVsTimeReno &
    exec xgraph -bb -tk -x time -y queue temp.queue &
    exit 0
}

proc plotWindow {tcpSource file} {
    global ns
    set time 0.01
    set now [$ns now]
    set cwnd [$tcpSource set cwnd_]
    puts $file "$now $cwnd"
    $ns at [expr $now+$time] "plotWindow $tcpSource $file"
}

# Узлы сети:
set N 5

```

Рис. 3.1: Скрипт модели с дисциплиной RED

Вторая часть скрипта модели с дисциплиной RED. (рис. 3.2)

```

# Узлы сети:
set N 5
for {set i 1} {$i < $N} {incr i} {
    set node_($i) [$ns node]
}
set node_(r1) [$ns node]
set node_(r2) [$ns node]

# Соединения:
$ns duplex-link $node_(s1) $node_(r1) 10Mb 2ms DropTail
$ns duplex-link $node_(s2) $node_(r1) 10Mb 3ms DropTail
$ns duplex-link $node_(r1) $node_(r2) 1.5Mb 20ms RED
$ns queue-limit $node_(r1) $node_(r2) 25
$ns queue-limit $node_(r2) $node_(r1) 25
$ns duplex-link $node_(s3) $node_(r2) 10Mb 4ms DropTail
$ns duplex-link $node_(s4) $node_(r2) 10Mb 5ms DropTail

# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Reno $node_(s1) TCPSink $node_(s3) 0]
$tcp1 set window 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]
$tcp2 set window 15
set ftp1 [$tcp1 attach-source FTP]
set ftp2 [$tcp2 attach-source FTP]

# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeReno w]
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1];
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout;

# Мониторинг очереди:
set redq [$ns link $node_(r1) $node_(r2)] queue]
set tchan [open all.q w]
$redq trace curq_
$redq trace ave_
$redq attach $tchan_

# Добавление at-событий:
$ns at 0.0 "$ftp1 start"
$ns at 1.1 "plotWindow $tcp1 $windowVsTime"
$ns at 3.0 "$ftp2 start"
$ns at 10 "finish"

# запуск модели
$ns run

```

Рис. 3.2: Скрипт модели с дисциплиной RED

График динамики размера окна TCP(сверху) и график динамики длины очереди и средней длины очереди(снизу) при типе протокола TCP Reno на узле s1. (рис. 3.3).

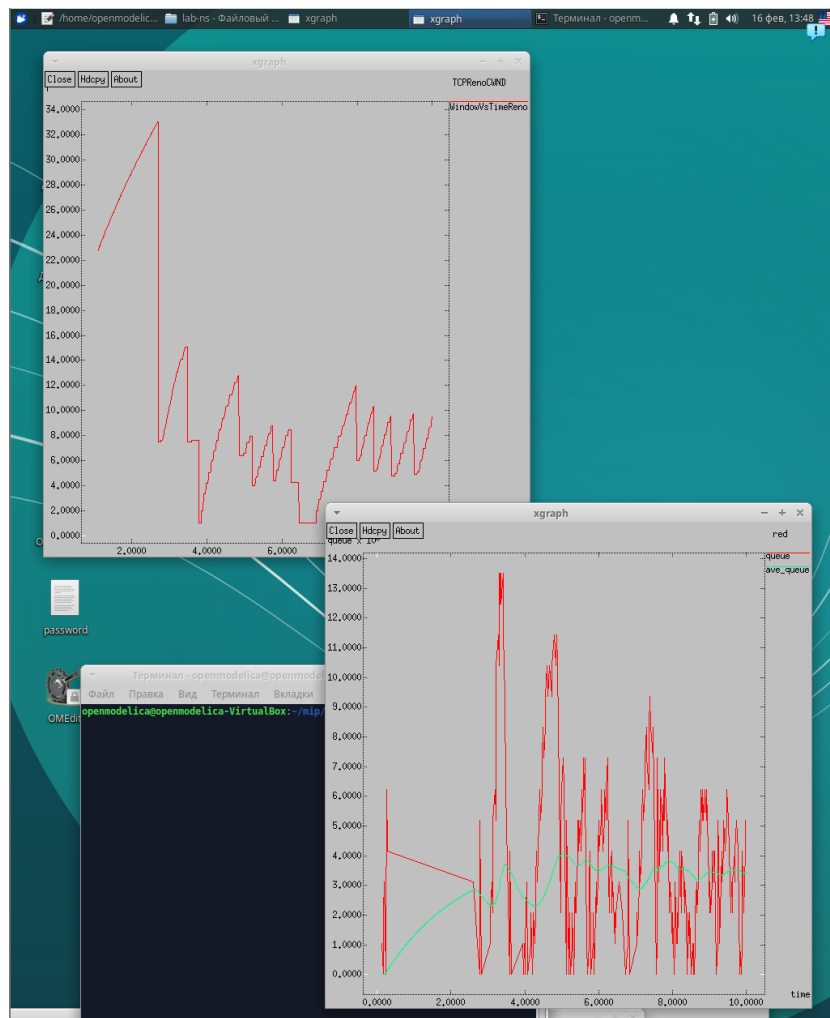


Рис. 3.3: График динамики размера окна TCP(сверху) и график динамики длины очереди и средней длины очереди(снизу) при типе протокола TCP Reno на узле s1

3.2 Изменения в модели на узле s1 типа протокола TCP с Reno на NewReno, затем на Vegas.

Скрипт изменений на узле s1 типа протокола TCP с Reno на Newreno. (рис. 3.4).

```
# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Newreno $node_(s1) TCPSink $node_(s3) 0]
$tcp1 set window_ 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]
$tcp2 set window_ 15
set ftp1 [$tcp1 attach-source FTP]
set ftp2 [$tcp2 attach-source FTP]
```

Рис. 3.4: Скрипт изменений на узле s1 типа протокола TCP с Reno на Newreno

График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP NewReno на узле s1. (рис. 3.5).

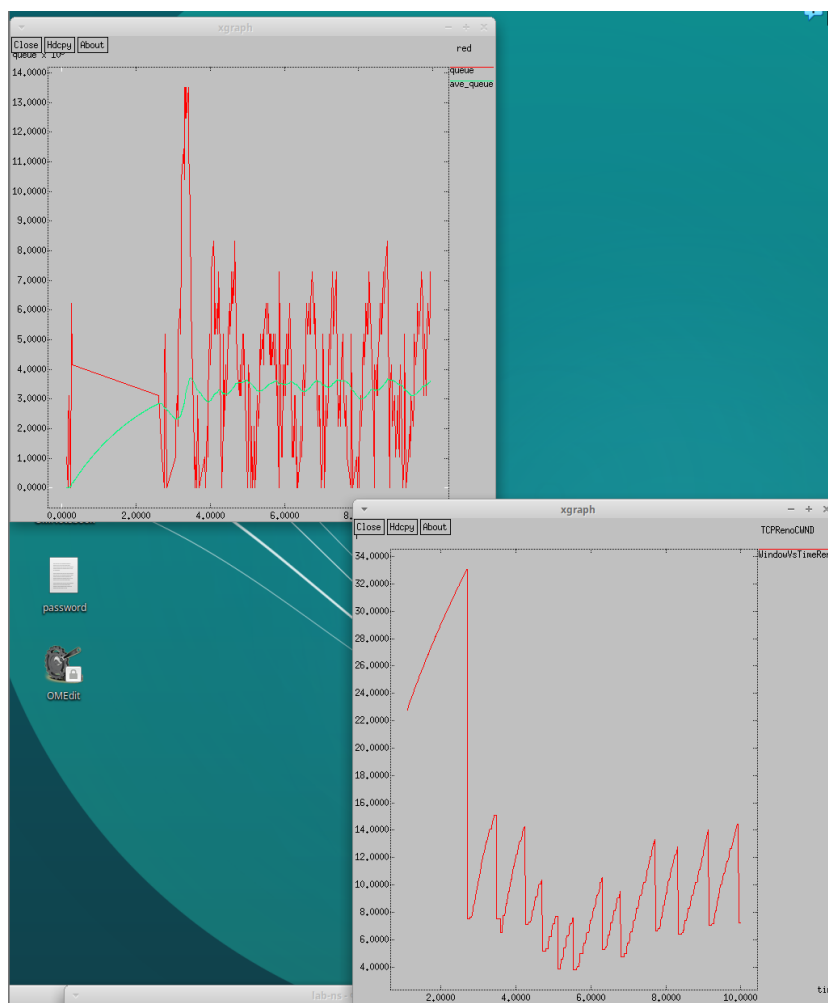


Рис. 3.5: График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP NewReno на узле s1

TCP NewReno: Улучшенная версия Reno, устраняющая его недостатки:

- Улучшенный Fast Recovery. TCP NewReno остается в режиме восстановления после первой потери пакета и корректно обрабатывает несколько потерянных пакетов за один цикл передачи.
- Более точный механизм обнаружения потерь и адаптации скорости.

Скрипт изменений на узле s1 типа протокола TCP с Reno на Vegas. (рис. 3.6).

```
# Агенты и приложения:  
set tcp1 [$ns create-connection TCP/Vegas $node_(s1) TCPSink $node_(s3) 0]  
$tcp1 set window_ 15  
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]  
$tcp2 set window_ 15  
set ftp1 [$tcp1 attach-source FTP]  
set ftp2 [$tcp2 attach-source FTP]
```

Рис. 3.6: Скрипт изменений на узле s1 типа протокола TCP с Reno на Vegas

График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP Vegas на узле s1. (рис. 3.7).

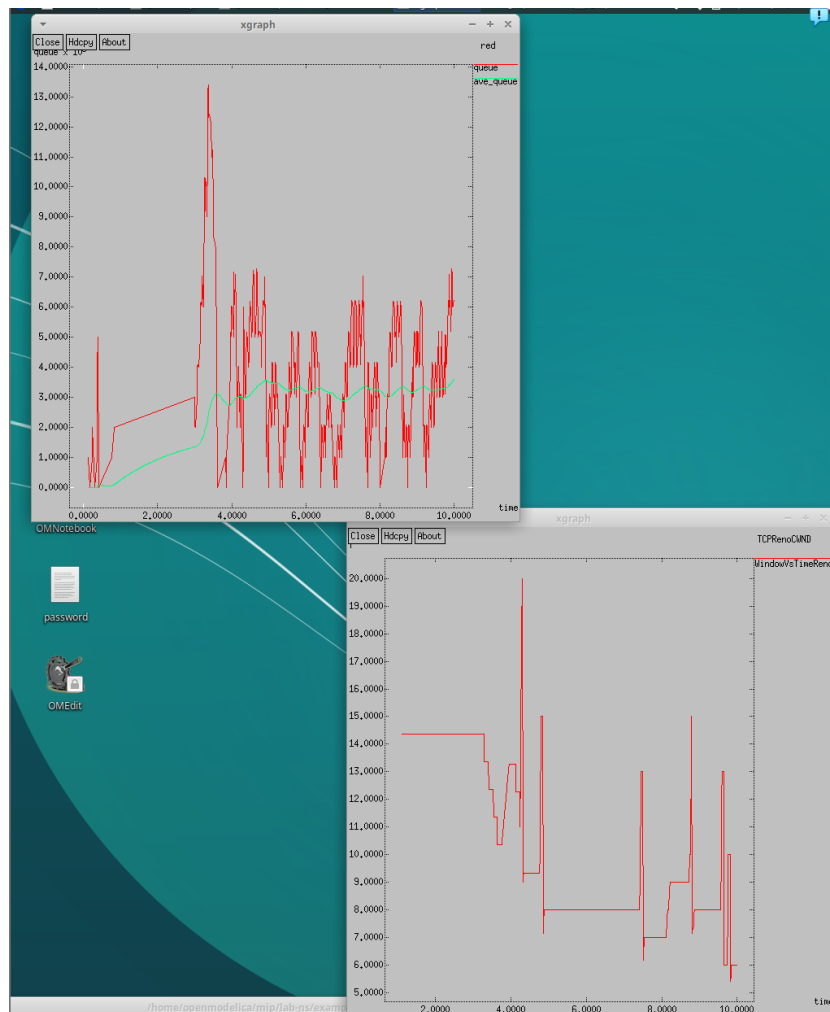


Рис. 3.7: График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP Vegas на узле s1

TCP Vegas: TCP Vegas использует другой подход:

- Оценивает задержку пакетов вместо того, чтобы просто реагировать на потери.
- Контролирует перегрузку до ее возникновения, измеряя разницу между ожидаемой и реальной скоростью передачи.
- Более гладкая регулировка CWND, без резких изменений, как в Reno/NewReno.

3.3 Изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

Изменение процедуры finish. (рис. 3.8).

```
proc finish {} {
    global tchan_
    # подключение кода AWK:
    set awkCode {
        {
            if ($1 == "Q" && NF>2) {
                print $2, $3 >> "temp.q";
                set end $2
            }
            else if ($1 == "a" && NF>2)
                print $2, $3 >> "temp.a";
        }
    }
    set f [open temp.queue w]
    puts $f "TitleText: RED"
    puts $f "Device: Postscript"
    puts $f "0.Color: Orange"
    puts $f "1.Color: Purple"
    if { [info exists tchan_] } {
        close $tchan_
    }
    exec rm -f temp.q temp.a
    exec touch temp.a temp.q
    exec awk $awkCode all.q
    puts $f "\nQ"
    exec cat temp.q >@ $f
    puts $f "\n\avg_Q"
    exec cat temp.a >@ $f
    close $f
    # Запуск xgraph с графиками окна TCP и очереди:
    exec xgraph -bg white -bb -tk -x time -y Razmer "TCPReNoCWND" WindowVsTimeReno &
    exec xgraph -bg white -bb -tk -x time -y QUEUE temp.queue &
    exit 0
}
```

Рис. 3.8: Изменение процедуры finish

Изменение мониторинга размера окна TCP. (рис. 3.9).

```
# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeReno w]
puts $windowVsTime "0.Color: Black"
puts $windowVsTime "\"dinamica_raz_okna"
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1];
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout;
```

Рис. 3.9: Изменение мониторинга размера окна TCP

Результаты изменений отображения окон с графиками. (рис. 3.10).

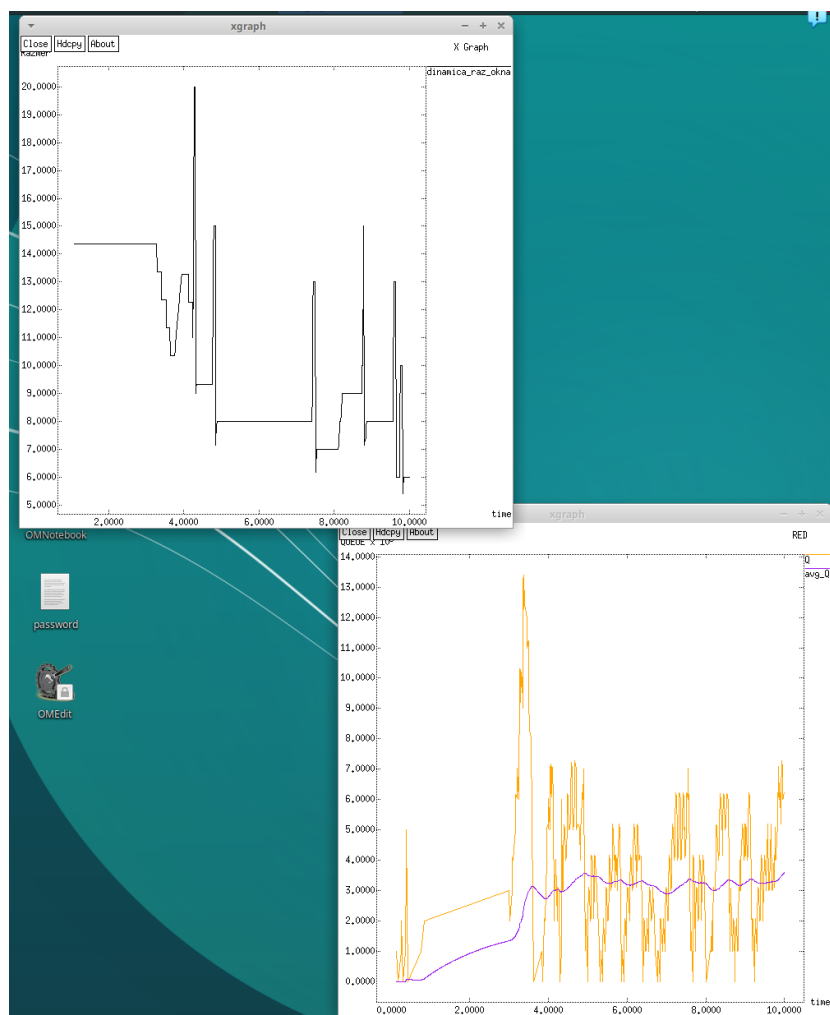


Рис. 3.10: Результаты изменений отображения окон с графиками

4 Выводы

Мы исследовали протокол TCP и алгоритм управления очередью RED.

Список литературы

Кулябов Д.С. Королькова А.В. Лабораторная работа №2. Исследование протокола ТСП и алгоритма управления очередью RED. [Электронный ресурс].